

A Appendix

A.1 Evaluation Metrics

1. Time Decrease (ΔT) Quantifies the relative latency reduction when using speculative planning. Specifically, for each task, we calculate the total time taken to complete the task under SP (T^{SP}) and sequential planning (T^{seq}). The percentage time saving is then calculated as:

$$\Delta \text{Time} = \frac{1}{N} \sum_{i=1}^N \left(1 - \frac{T_i^{\text{SP}}}{T_i^{\text{seq}}} \right) \times 100\%$$

where N is the total number of tasks. Positive values indicate time savings.

2. Cost and Token Increase Token increase captures the *redundant* resources consumed by speculative planning as a result of incorrect speculation. While the primary goal of speculative planning is to reduce latency and enhance efficiency, it still incurs a minimum cost. Even with perfect accuracy in speculative planning (*i.e.*, the approximation agent makes correct predictions for every step), the cost must at least include the *baseline cost* — the minimal overhead associated with sequential planning of both the target and approximation agents. Thus, we define the baseline for token increase as the sum of the planning costs for the target agent and approximation agent operating sequentially, which represents the least amount of resources required for completing the task with speculation. Any increase in token usage beyond this baseline reflects the additional overhead introduced by speculative failures, capturing the waste generated from incorrect predictions by the approximation agent.

Since the cost per token can differ for *prompt tokens* and *generation tokens*, we separately evaluate these components and also provide a combined metric for *total cost increase*.

Prompt Token Increase (ΔP) This measures the increase in prompt tokens due to redundant prompt usage in SP. Let $p_i^{\text{seq}} = p_i^{\text{target}} + p_i^{\text{approx}}$ be the baseline prompt tokens (the sum of target and approximation prompts under sequential execution), and p_i^{SP} be the prompt tokens used under SP (parallel execution). The prompt token increase is calculated as:

$$\Delta P = \frac{1}{N} \sum_{i=1}^N \left(\frac{p_i^{\text{SP}}}{p_i^{\text{seq}}} - 1 \right) \times 100\% \quad (3)$$

Generation Token Increase (ΔG) This metric captures the increase in generation tokens caused by incorrect speculation. Let $g_i^{\text{seq}} = g_i^{\text{target}} + g_i^{\text{approx}}$ represent the baseline generation tokens (the sum of target and approximation generations under sequential execution), and g_i^{SP} the generation tokens used under SP. The generation token increase is calculated as:

$$\Delta G = \frac{1}{N} \sum_{i=1}^N \left(\frac{g_i^{\text{SP}}}{g_i^{\text{seq}}} - 1 \right) \times 100\% \quad (4)$$

Total Cost Increase (ΔCost) This metric quantifies the relative increase in cost between speculative planning (SP) and sequential planning (seq) strategies. Specifically, we define the total cost increase as:

$$\Delta \text{Cost} = \frac{1}{N} \sum_{i=1}^N \left(\frac{PC_i^{\text{SP}} + GC_i^{\text{SP}}}{PC_i^{\text{seq}} + GC_i^{\text{seq}}} - 1 \right) \times 100\% \quad (5)$$

where PC_i^{SP} and PC_i^{seq} represent the prompt costs of task i for SP and sequential strategies, and GC_i^{SP} and GC_i^{seq} represent the corresponding generation costs. These costs are computed as follows:

$$PC_i^{\text{SP}} = ap_i^{\text{SP}} \cdot \text{cost}_{\text{ap}} + tp_i^{\text{SP}} \cdot \text{cost}_{\text{tp}} \quad (6)$$

$$PC_i^{\text{seq}} = ap_i^{\text{seq}} \cdot \text{cost}_{\text{ap}} + tp_i^{\text{seq}} \cdot \text{cost}_{\text{tp}} \quad (7)$$

$$GC_i^{\text{SP}} = ag_i^{\text{SP}} \cdot \text{cost}_{\text{ag}} + tg_i^{\text{SP}} \cdot \text{cost}_{\text{tg}} \quad (8)$$

$$GC_i^{\text{seq}} = ag_i^{\text{seq}} \cdot \text{cost}_{\text{ag}} + tg_i^{\text{seq}} \cdot \text{cost}_{\text{tg}} \quad (9)$$

Here, ap_i^{SP} and tp_i^{SP} represent the number of prompt tokens for the approximation and target in the SP strategy at task i , and ap_i^{seq} and tp_i^{seq} represent the number of prompt tokens for the approximation and target in the sequential planning at task i . Similarly, ag_i^{SP} , tg_i^{SP} , ag_i^{seq} and tg_i^{seq} represent the corresponding number of generation tokens. For cost , cost_{ap} and cost_{tp} are the costs for the approximation and target API for prompt tokens, while cost_{ag} and cost_{tg} are the corresponding costs for generation tokens.

By computing the total cost increase, we can quantify the trade-off between speculative planning and sequential planning in terms of the associated computational costs, helping to assess the efficiency of the proposed approach.

3. Absolute Time Ratio ($T(\times)$) This metric quantifies the ratio of the absolute time in the current configuration compared to the baseline configuration (fix $k = 2$), as it represents the minimal non-trivial setting that offers meaningful acceleration with minimal resource cost. It is calculated as:

$$T(\times) = \frac{T^{\text{current}}}{T^{k=2}}$$

4. Absolute Cost and Token Ratio This metric measures the ratio of the absolute token usage and cost in the current configuration compared to the baseline configuration (fix $k = 2$).

Absolute Prompt Token Ratio ($P(\times)$)

$$P(\times) = \frac{p_i^{\text{current}}}{p_i^{k=2}}$$

Absolute Generation Token Ratio ($G(\times)$)

$$G(\times) = \frac{g_i^{\text{current}}}{g_i^{k=2}}$$

Absolute Cost Ratio ($\text{Cost}(\times)$)

$$\text{Cost}(\times) = \frac{C^{\text{current}}}{C^{k=2}}$$

5. Concurrency Dynamics Evaluating the concurrency behavior of speculative planning (SP) is essential for understanding its real-world deployability. Modern systems often impose limits on simultaneous API requests due to infrastructure constraints or cost-efficiency considerations. Excessive concurrency can lead to API throttling, degraded performance, or instability, making it critical to monitor and control parallel request patterns. To quantify this behavior, we introduce two metrics.

Average Max Concurrent API Calls ($\overline{MC}$) For each task i , MC_i denotes the peak number of overlapping API requests during speculative execution. $\overline{MC}$ is calculated as:

$$\overline{MC} = \frac{1}{N} \sum_{i=1}^N MC_i \quad (10)$$

where N is the number of tasks. This metric reflects how aggressively a system exploits parallelism and whether it respects concurrency constraints under varying speculative policies.

Average Speculative Step ($\overline{K}$) In addition to peak concurrency ($\overline{MC}$), which captures brief spikes of load, we aim to characterize the *average* concurrency of the planning process over the planning process, as it correlates with sustained system pressure. However, the instantaneous concurrency varies continuously over time. Precisely computing the time-averaged number of concurrent API calls would require fine-grained logs of every call's start and end time, which is often impractical. To address this, we use the average speculative step $\overline{K}$ as a proxy for the true average concurrency. Let M be the total number of planning episodes, and k_i be the speculative steps used in episode i . We define

$$\overline{K} = \frac{1}{M} \sum_{i=1}^M k_i \quad (11)$$

Because each target-agent computation is relatively long-running, the k_i speculative calls in episode i tend to overlap in time. In practice, this means the average concurrency during episode i is effectively k_i . Averaging these values over all episodes therefore yields a meaningful estimate of the sustained concurrency.

A.2 Actual Performance

Table 4: Performance Comparison of Dynamic k vs. Fixed k Speculative Planning on **OpenAGI** benchmark with **GPT** backbone

Mode	Direct-ReACT (Setting 1)						CoT-MAD (Setting 2)					
	ΔT (%)	ΔP (%)	ΔG (%)	ΔCost (%)	$\overline{MC}$	$\overline{K}$	ΔT (%)	ΔP (%)	ΔG (%)	ΔCost (%)	$\overline{MC}$	$\overline{K}$
Fix ($k=2$)	18.00	39.47	20.71	33.94	3.00	2.00	22.35	31.57	27.78	29.81	3.00	2.00
Fix ($k=4$)	25.06	114.94	47.88	94.89	4.95	4.00	35.62	83.78	74.46	79.38	4.97	4.00
Fix ($k=6$)	25.88	212.93	90.41	176.25	6.32	6.00	37.21	145.42	122.01	134.38	6.44	6.00
Dyn. ($\tau=0.5$)	15.68	16.40	8.23	14.04	4.33	1.78	23.12	21.20	17.83	19.63	4.44	2.10
Dyn. ($\tau=0.8$)	19.43	30.98	17.05	26.96	4.41	2.33	30.09	33.51	29.41	31.61	4.73	2.67
Dyn. ($\tau=0.9$)	20.94	39.83	19.60	34.00	4.56	2.47	32.86	43.35	38.18	40.95	5.22	3.03
Dyn. ($\tau=0.95$)	22.31	51.36	27.00	44.22	4.75	2.88	34.24	62.51	56.14	59.53	5.64	3.58
Dyn. ($\tau=0.99$)	24.85	79.74	42.36	68.71	5.24	3.59	35.62	82.76	73.14	78.26	5.87	4.06
Dyn. (offset=1)	23.47	49.84	26.86	43.14	4.86	2.75	31.62	41.65	36.50	39.26	4.95	2.80
Dyn. (offset=2)	25.26	82.55	37.88	69.36	5.13	3.47	37.09	66.52	58.98	62.99	5.60	3.99

Table 5: Performance Comparison of Dynamic k vs. Fixed k Speculative Planning on **OpenAGI** benchmark with **DeepSeek** backbone

Mode	Direct-ReACT (Setting 3)						CoT-MAD (Setting 4)					
	ΔT (%)	ΔP (%)	ΔG (%)	ΔCost (%)	$\overline{MC}$	$\overline{K}$	ΔT (%)	ΔP (%)	ΔG (%)	ΔCost (%)	$\overline{MC}$	$\overline{K}$
Fix ($k=2$)	19.68	34.25	28.23	30.69	3.00	2.00	24.91	36.17	30.19	32.98	3.00	2.00
Fix ($k=4$)	29.19	109.41	90.92	95.15	4.92	4.00	34.22	99.61	86.99	90.01	4.94	4.00
Fix ($k=6$)	30.34	171.35	134.69	142.50	6.33	6.00	36.25	144.87	128.64	128.71	6.59	6.00
Dyn. ($\tau=0.5$)	16.88	20.54	17.16	18.74	3.89	1.69	17.03	21.90	17.58	19.99	3.78	1.54
Dyn. ($\tau=0.8$)	19.60	32.28	27.51	29.28	4.20	2.08	25.10	41.30	35.03	37.84	4.57	2.31
Dyn. ($\tau=0.9$)	23.84	47.71	40.77	42.58	4.45	2.58	27.67	52.66	45.25	48.14	4.69	2.65
Dyn. ($\tau=0.95$)	26.25	52.32	48.18	48.13	4.78	2.83	30.87	60.91	53.43	55.88	5.12	3.12
Dyn. ($\tau=0.99$)	28.70	87.02	75.24	75.92	5.46	3.85	32.84	84.59	74.42	77.30	5.29	3.75
Dyn. (offset=1)	25.07	46.30	40.92	42.40	4.39	2.58	29.99	49.08	42.90	45.41	4.48	2.80
Dyn. (offset=2)	29.58	78.31	68.64	69.90	4.99	3.53	35.24	76.23	66.90	69.92	5.38	3.85

Table 6: Performance Comparison of Dynamic k vs. Fixed k Speculative Planning on **Travel Planner** benchmark with **GPT** backbone

Mode	Direct-ReACT (Setting 1)						CoT-MAD (Setting 2)					
	ΔT (%)	ΔP (%)	ΔG (%)	ΔCost (%)	$\overline{MC}$	$\overline{K}$	ΔT (%)	ΔP (%)	ΔG (%)	ΔCost (%)	$\overline{MC}$	$\overline{K}$
Fix ($k=2$)	15.36	64.52	26.86	61.91	3.00	2.00	10.97	55.46	24.91	54.77	3.00	2.00
Fix ($k=4$)	23.48	215.76	73.93	205.98	5.00	4.00	14.53	130.46	51.66	128.93	4.62	4.00
Fix ($k=6$)	24.27	364.01	109.91	346.71	6.99	6.00	12.75	159.45	55.37	157.74	5.37	6.00
Dyn. ($\tau=0.5$)	10.47	31.96	15.28	30.74	4.28	1.46	5.61	27.46	8.24	27.02	4.09	1.30
Dyn. ($\tau=0.8$)	15.52	54.22	21.96	51.83	4.60	1.88	5.65	35.10	12.49	34.57	4.00	1.42
Dyn. ($\tau=0.9$)	18.71	72.84	29.59	69.56	4.96	2.21	9.07	55.08	22.06	54.26	4.03	1.79
Dyn. ($\tau=0.95$)	19.94	95.01	37.10	90.72	5.19	2.55	12.74	57.54	22.35	56.65	4.00	2.10
Dyn. ($\tau=0.99$)	21.26	137.90	50.41	132.21	5.32	3.10	14.54	93.63	41.01	92.49	4.58	2.94
Dyn. (offset=1)	12.55	54.95	22.22	52.13	4.31	2.30	11.86	62.65	26.30	61.96	4.08	2.16
Dyn. (offset=2)	15.20	89.08	30.84	84.24	4.91	3.22	12.75	105.28	43.65	104.16	4.33	2.98

Figure 4 visualizes how different speculative planning configurations in Setting 1 of the OpenAGI benchmark impact prompt token increase, generation token increase, and the corresponding reduction in execution time.

On OpenAGI benchmark: In **setting 1**, Dyn. (offset = 2) achieves a time reduction comparable to the fastest fixed configuration (Fix ($k = 6$)), while reducing redundant cost by **60.65%**. Under **Setting 2**, Dyn. ($\tau = 0.5$) achieves a 23.12% latency reduction while incurring only 19.63% unnecessary cost—saving 34.15% redundant cost compared to the most cost-efficient fixed configuration (Fix, $k = 2$), and delivering better speed. The Dyn. (offset = 2) configuration matches the time savings of the fastest fixed strategy (Fix, $k = 6$), while reducing redundant cost by **53.13%**. Under **setting 3**, Dyn. (offset = 2) delivers time savings comparable to the fastest fixed configuration (Fix, $k = 6$), while reducing redundant cost by **50.95%**. Under **setting 4**, Dyn. (offset = 2) achieves a 97.21% latency reduction relative to the fastest fixed baseline (Fix $k = 6$), while concurrently reducing redundant cost by approximately 45.68%.

On Travel Planner benchmark: In **setting 1**, Dyn. ($\tau = 0.8$) achieves slightly faster time reduction than Fix ($k = 2$), while reducing redundant cost by 16.28%. Under **setting 2**, Dyn. ($\tau = 0.99$) achieves lower latency than both Fix ($k = 4$) and Fix ($k = 2$), while separately reducing redundant cost by 28.26% and 41.37% respectively.

Experiment results also demonstrate that DSP achieves significantly more efficient concurrency utilization compared to fixed k baselines. DSP achieves 40% lower sustained system pressure($\bar{K}$) in its fastest configurations (Dyn. ($\tau = 0.99$), Dyn. (offset = 2)), compared to the fastest fixed- k setting (Fix ($k = 6$)), while maintaining comparable latency reduction. This demonstrates that dynamic policies can minimize persistent system load without sacrificing acceleration performance. All dynamic variants maintain $\overline{MC}$ below 6, outperforming the fastest Fix ($k = 6$) configuration.

555 A.3 Online Warmup

In our experiment, we begin with a randomly initialized DistilBERT model at the initial stage of every setting and its predictions are naturally zero at the start. This results in the entire pipeline executing the target agent in a sequential manner during warm-up tasks without any additional cost increase. To expedite the reduction of warmup time, we start collecting training episodes online as the pipeline runs. Once enough episodes are gathered to meet the predefined batch size, the model undergoes its first training update.

Our analysis across different settings reveals that the warmup duration varies depending on the API backbone. When using GPT-4.1-mini, it typically requires around 3–5 tasks to warm up, as GPT-4.1-mini tends to generate longer sequences to complete each task. It enables faster collection of a sufficient number of training episodes. In contrast, when using DeepSeek-series models, the warmup phase is slower, requiring approximately 8–10 tasks to collect enough training data. This discrepancy highlights the differences in the response times of different APIs and the resulting impact on the warmup efficiency.

569 A.4 DSP Cost Broken-Down

The cost breakdown data for Fix and Dynamic Speculative Planning across different asymmetry parameters τ and different offset is presented in Table.7. In both DSP settings, the cost structure maintains the same general pattern observed in fixed- k Speculative Planning, with the majority of tokens being consumed by target agent operations in input prompt; but τ parameter effectively controls the aggressiveness of speculation, showing clear patterns in cost reduction:

At the most conservative setting where $\tau = 0.5$, the DSP reduces the average redundant token consumption from +10,748.3 to +672.2 in setting 1 and from +20,424.0 to +8,508.7 redundant tokens in setting 2, respectively, comparing with the most conservative fixed- k setting where $k = 2$. *This represents an around 71% reduction in redundant token consumption.* At the most aggressive setting where $\tau = 0.99$, the DSP also reduces the average total token consumption from +52,664.6 to +14,452.6 in setting 1 and from +96,927.3 to +50,977.8 in setting 2, respectively, comparing with the most aggressive fixed- k setting where $k = 6$. *This also represents an around 50% reduction in redundant token consumption.* The graduated scaling of costs with the τ parameter provides practitioners with fine-grained control over the efficiency-cost balance, enabling precise system calibration that fixed- k approaches cannot match.

585 A.5 Biased k Prediction *v.s.* k with Biased Offset

Tunable trade-off control The τ hyperparameter provides fine-grained control over the latency-cost trade-off by enabling three operational modes, defining a smooth continuum of operating points, forming a Pareto frontier between latency and token cost. This flexibility allows practitioners to adjust system behavior dynamically to meet varying operational requirements and economic constraints.

- **High-Performance Mode** ($\tau = 0.99$, $\tau = 0.95$): It provides over 90% of the time savings compared to fastest setting (even exceeding its performance) while saving 30% total cost.
- **Balanced Mode** ($\tau = 0.9$, $\tau = 0.8$): It achieves over 80% of latency reduction, while on average halves total cost compared to the fastest setting.
- **Economy Mode** ($\tau = 0.5$): It maintains non-trivial acceleration while reducing cost overhead compared to the most cost-efficient fixed setting.

All three modes are supported within a unified DSP framework by simply tuning τ , without altering the algorithm. This tunability allows DSP to meet different application requirements. In DSP, τ provides a smooth continuum of operating points, effectively generating a Pareto curve of latency vs.

Table 7: Speculative Planning cost broken-down on OpenAGI benchmark with GPT backbone**(a) Direct-ReAct (Setting 1)**

Plan Type	Approx Prompt	Approx Generation	Target Prompt	Target Generation	Total Prompt	Total Generation	Total Token
Normal Plan	6040.3	108.6	12715.6	1906.7	18755.9	2015.3	20771.2
Fix ($k=2$)	10759.8	165.7	18162.7	2431.2	28922.5	2596.9	31519.4
Fix ($k=4$)	19894.5	281.9	26610.1	2963.1	46504.6	3245.1	49749.7
Fix ($k=6$)	34124.0	369.2	35079.7	3862.9	69203.7	4232.1	73435.8
Dyn. ($\tau=0.5$)	6457.7	115.0	13022.8	1965.6	19480.5	2081.6	21443.4
Dyn. ($\tau=0.8$)	7454.8	130.9	14348.4	1994.6	21803.1	2125.5	23928.6
Dyn. ($\tau=0.9$)	8745.6	148.9	16062.6	2185.7	24808.2	2334.6	27142.8
Dyn. ($\tau=0.95$)	9226.2	154.7	16407.2	2176.9	25633.4	2331.6	27965.0
Dyn. ($\tau=0.99$)	12307.0	195.1	20140.0	2581.7	32447.0	2776.8	35223.8
Dyn. (offset = 1)	9576.3	163.9	17381.4	2293.0	26957.7	2456.9	29414.7
Dyn. (offset = 2)	12999.8	206.1	20254.3	2457.7	33254.1	2663.8	35917.9
Redundant Fix ($k=2$)	+4719.5	+57.1	+5447.1	+524.5	+10166.6	+581.6	+10748.3
Redundant Fix ($k=4$)	+13854.2	+173.3	+13894.4	+1056.5	+27748.7	+1229.8	+28978.5
Redundant Fix ($k=6$)	+28083.7	+260.6	+22364.1	+1956.2	+50447.8	+2216.8	+52664.6
Redundant Dyn. ($\tau=0.5$)	+417.4	+6.4	+307.2	+58.9	+724.6	+66.3	+767.2
Redundant Dyn. ($\tau=0.8$)	+1414.5	+22.3	+1632.8	+87.9	+3047.2	+110.2	+3157.4
Redundant Dyn. ($\tau=0.9$)	+2705.3	+40.3	+3347.0	+279.0	+6052.3	+319.3	+6371.6
Redundant Dyn. ($\tau=0.95$)	+3185.9	+46.1	+3691.6	+270.2	+6877.5	+316.3	+7193.8
Redundant Dyn. ($\tau=0.99$)	+6266.7	+86.5	+7424.4	+675.0	+13691.1	+761.5	+14452.6
Redundant Dyn. (offset = 1)	+3536.0	+55.3	+4465.8	+386.3	+8201.8	+441.6	+8643.5
Redundant Dyn. (offset = 2)	+6959.5	+97.5	+7538.7	+551.0	+14498.2	+648.5	+15146.7

(b) CoT-MAD (Setting 2)

Plan Type	Approx Prompt	Approx Generation	Target Prompt	Target Generation	Total Prompt	Total Generation	Total Token
Normal Plan	5422.5	1701.3	35968.4	7332.8	41390.9	9034.0	50424.9
Fix ($k=2$)	8158.9	2435.4	50373.5	9881.1	58532.4	12316.5	70848.9
Fix ($k=4$)	13799.2	3884.8	70283.3	13253.9	84082.5	17138.7	101221.2
Fix ($k=6$)	26065.3	5466.1	97668.0	18152.8	123733.3	23618.9	147352.2
Dyn. ($\tau=0.5$)	6429.6	1946.9	42200.3	8356.7	48629.9	10303.7	58933.6
Dyn. ($\tau=0.8$)	7671.9	2330.3	48703.2	9577.7	56375.1	11908.0	68283.1
Dyn. ($\tau=0.9$)	8682.5	2590.9	52586.8	10255.2	61269.3	12846.1	74115.5
Dyn. ($\tau=0.95$)	10417.5	3043.1	58605.2	11353.2	69022.7	14396.4	83419.1
Dyn. ($\tau=0.99$)	13857.5	3848.7	70281.1	13415.5	84138.5	17264.2	101402.7
Dyn. (offset = 1)	8978.2	2699.9	54749.3	10715.0	63727.4	13414.9	77142.3
Dyn. (offset = 2)	11256.8	3276.3	61864.3	11828.1	73121.1	15104.4	88225.6
Redundant Fix ($k=2$)	+2736.4	+734.1	+14405.1	+2548.3	+17141.5	+3282.4	+20424.0
Redundant Fix ($k=4$)	+8376.7	+2183.5	+34314.9	+5921.2	+42691.6	+8104.7	+50796.3
Redundant Fix ($k=6$)	+20642.8	+3764.9	+61699.6	+10820.0	+82342.4	+14584.9	+96927.3
Redundant Dyn. ($\tau=0.5$)	+1007.1	+245.6	+6231.9	+1023.9	+7239.0	+1269.7	+8508.7
Redundant Dyn. ($\tau=0.8$)	+2249.4	+629.0	+12734.8	+2244.9	+14984.2	+2874.0	+17858.2
Redundant Dyn. ($\tau=0.9$)	+3260.0	+889.6	+16618.4	+2922.4	+19878.4	+3812.1	+23690.6
Redundant Dyn. ($\tau=0.95$)	+4995.0	+1341.8	+22636.8	+4020.4	+27631.8	+5362.4	+32994.2
Redundant Dyn. ($\tau=0.99$)	+8435.0	+2147.4	+34312.7	+6082.7	+42747.6	+8230.2	+50977.8
Redundant Dyn. (offset = 1)	+3555.7	+998.6	+18780.9	+3382.2	+22336.5	+4380.9	+26717.4
Redundant Dyn. (offset = 2)	+5834.3	+1575.0	+25895.9	+4495.3	+31730.2	+6070.4	+37800.7

token cost from which one can choose according to preferences. As an interpretable hyperparameter, τ offers clear deployment guidance: higher values promote larger k predictions, while lower values encourage conservative steps, as detailed in Section 4.3. As shown in Table 4, Table 5, and Table 6, the average speculation step $\bar{K}$ increases with τ , confirming its practical and theoretical role in governing speculative planning behavior.

Biased k Prediction v.s. k with Biased Offset We examine two distinct approaches for controlling k : training-time bias adjustment through expectile regression and inference-time direct offset application. Both methods enable practitioners to navigate the latency-cost tradeoff, though with different optimization characteristics.

Applying a fixed positive offset to the unbiased predicted value of k offers a computationally efficient and conceptually straightforward approach to balancing acceleration and cost efficiency. In OpenAGI benchmark, DSP(offset=1) achieves a latency reduction comparable to DSP($\tau=0.95$). Similarly, DSP(offset=2) achieved 97% acceleration by the optimal fixed- k setting where $k=6$, while saving up to 28% cost. These results demonstrate that simple post-prediction adjustment yields robust performance improvements across both latency and cost metrics.

However, the principal advantage of biased- k prediction implemented through training-time expectile regression lies in its capacity for more granular control over the efficiency-cost continuum. Experimental results indicate that moderate training parameters ($\tau \in 0.8, 0.9$) delivers acceleration profiles that cannot be precisely matched by integer offset adjustments. These intermediate settings enable more nuanced optimization configurations that achieve reasonable latency reduction with substantially lower operational costs than their offset-based counterparts (offset $\in \{+1, +2\}$).

The choice between these approaches ultimately depends on the specific operational requirements and constraints. Training-time bias adjustment provides finer-grained control and potentially better optimization at intermediate points on the efficiency-cost spectrum, particularly valuable for applications with precise performance requirements. Conversely, inference-time offset application offers

624 implementation simplicity, immediate adjustability, and strong performance at discrete operating
625 points, making it suitable for deployments where straightforward configuration and rapid adaptation
626 are prioritized.

627 **A.6 Limitations**

628 While our Dynamic Speculative Planning framework demonstrates significant improvements in
629 balancing acceleration and cost efficiency, several important limitations warrant acknowledgment and
630 suggest directions for future research. First, our current implementation does not adaptively select or
631 optimize the approximation agent itself. The speed, accuracy, and availability characteristics of this
632 agent represent crucial factors in overall system performance. Different approximation agents may be
633 better suited for specific tasks or planning stages, and dynamically selecting from a pool of candidate
634 agents could further enhance both acceleration and cost-efficiency. Future work could explore meta-
635 learning approaches that automatically select or even generate task-specific approximation agents
636 based on observed performance patterns.

637 Second, our current rejection criteria for verifying speculation steps remain relatively strict, potentially
638 limiting achievable acceleration. Agent planning often admits multiple valid solution paths to
639 accomplish the same task, yet our verification mechanism may incorrectly reject valid alternative
640 approaches proposed by the approximation agent when they differ syntactically from the target agent’s
641 preferred approach. This high false negative rate in the rejection criteria unnecessarily constrains
642 parallelization opportunities. Future research could explore semantic verification mechanisms that
643 recognize functionally equivalent actions despite surface-level differences, or could implement more
644 sophisticated verification strategies that consider the downstream effects of actions rather than just
645 their immediate representation.

646 Finally, our evaluation focused primarily on two standard agent benchmarks; more extensive testing
647 across diverse application domains would provide greater confidence in the generalizability of our
648 findings.

649 Despite these limitations, our work represents a significant step toward making accelerated LLM-
650 based agents more practical for real-world deployment by addressing the critical tension between
651 latency reduction and cost efficiency while providing unprecedented user control over system behavior.

652 **A.7 Broader Impact**

653 Dynamic Speculative Planning contributes to the broader effort of making AI agents more practical
654 and accessible for real-time applications. By substantially reducing response latency while providing
655 fine-grained control over operational costs, our approach can help the deployment of LLM-based
656 agents in time-sensitive contexts. For examples: educational applications could benefit from more
657 responsive tutoring agents that provide immediate feedback while managing computational expenses
658 within institutional constraints; healthcare systems might leverage faster diagnostic support agents
659 without compromising accuracy or incurring prohibitive costs; customer service platforms could
660 deploy assistants with response times approaching human conversation pace.